UTN
Facultad Regional Córdoba

TUP

Tecnicatura Universitaria en Programación

PROGRAMACION III
BACK END

Anexo:

POO - SOLID

Anexo

2° Año – 3° Cuatrimestre

CC BY ND NC

Índice

INTRODUCCIÓN 2

Conceptos fundamentales de la POO 3
Clase 3
Objeto 4
Encapsulamiento 4
Abstracción 6
Herencia 8
Polimorfismo 10
Interfaces 13

Principios de Diseño SOLID 16

Principio de Responsabilidad Única (Single Responsibility Principle) 16
Principio Abierto/Cerrado (Open/Closed Principle) 18
Principio de Sustitución de Liskov (Liskov Substitution Principle) 20
Principio de Segregación de Interfaces (Interface Segregation Principle) 22
Principio de Inversión de Dependencias (Dependency Inversion Principle) 24

Conclusión 29

INTRODUCCIÓN

La **Programación Orientada a Objetos (POO)** es un paradigma de desarrollo de software en el que el diseño se basa en entidades llamadas **objetos**. En lugar de escribir código como una serie de instrucciones secuenciales (estilo procedimental), en POO pensamos en *modelar* elementos del problema a resolver en forma de objetos que poseen **propiedades** (datos) y **comportamientos** (operaciones) similares a los del mundo real. Java es uno de los lenguajes más populares que adopta el paradigma POO, lo que permite crear software modular, reutilizable y más fácil de mantener.

En esta guía abordaremos los **conceptos clave de la POO** (clase, objeto, encapsulamiento, abstracción, herencia, polimorfismo e interfaces) con explicaciones claras y ejemplos sencillos en código Java. Además, exploraremos los **principios de diseño SOLID** que ayudan a escribir código orientado a objetos de calidad. El objetivo es proporcionar una comprensión sólida y práctica de la POO en Java, utilizando un lenguaje amigable pero riguroso, orientado a estudiantes universitarios.

Conceptos fundamentales de la POO

La POO introduce una serie de conceptos fundamentales que amplían la forma tradicional de programar. Los más importantes son: **clase**, **objeto**, **encapsulamiento**, **abstracción**, **herencia**, **polimorfismo** e **interfaces**. A continuación, describiremos cada concepto con definiciones accesibles y ejemplos en Java.

Clase

Una **clase** es una especie de *plantilla* o modelo a partir del cual se crean objetos. En una clase se definen los **atributos** (datos que describen las características) y los **métodos** (funciones o comportamientos) que los objetos de esa clase tendrán. Podemos imaginar una clase como el plano arquitectónico de una casa: especifica qué *propiedades* tendrá (número de habitaciones, tamaño, color) y qué *acciones* se pueden hacer en ella (abrir puertas, encender luces), pero no es la casa en sí, sino la definición para construirla.

En Java, definir una clase implica escribir una estructura que incluya atributos y métodos. Por ejemplo, definamos una clase sencilla `Persona` con algunos atributos y un método:

```java
// Definición de la clase Persona
public class Persona {
    // Atributos (propiedades) de la persona
    String nombre;
    int    edad;

    // Método (comportamiento) de la persona
    void saludar() {
        System.out.println("Hola, mi nombre es " + nombre + " y tengo " + edad + " años.");
    }
}
```

Imagen 1: Elaboración Propia

En este ejemplo, `Persona` tiene dos atributos (`nombre` y `edad`) y un método `saludar()` que imprime un saludo personalizado. La clase actúa como un **molde**: define qué datos y acciones *pueden* tener las personas, pero no representa a ninguna persona en particular hasta que la *instanciemos* como un objeto en el programa.

Objeto

Un **objeto** es una **instancia de una clase**. Siguiendo el ejemplo anterior, si la clase es el plano de una casa, el objeto sería la casa construida según ese plano. El objeto toma la estructura definida por la clase, pero con valores concretos en sus atributos.

Cuando creamos un objeto en Java, usamos la palabra clave `new` para instanciar la clase. Veamos cómo crear y usar un objeto de la clase `Persona`:

```
public class Ejemplo {
    public static void main(String[] args) {
        // Crear (instanciar) un objeto de la clase Persona
        Persona persona1 = new Persona();

        // Asignar valores a los atributos del objeto
        persona1.nombre = "Alice";
        persona1.edad = 30;

        // Invocar un método del objeto
        persona1.saludar();
        // Salida esperada: "Hola, mi nombre es Alice y tengo 30 años."
    }
}
```

Imagen 2: Elaboración Propia

En el código anterior, `persona1` es un objeto de tipo Persona. Cada objeto tiene su **estado interno** (valores actuales de sus atributos) y puede ejecutar sus métodos. Podemos crear múltiples objetos de la clase `Persona`, cada uno con valores distintos en sus atributos, representando así distintas personas. Por ejemplo, Persona `persona2 = new Persona(); persona2.nombre = "Bob"; persona2.edad = 25;` crearía otro objeto persona con nombre "Bob" y edad 25, independiente de `persona1`.

Encapsulamiento

El **encapsulamiento** es el mecanismo que permite agrupar los datos y los métodos que operan sobre esos datos dentro de una unidad (la clase), y **ocultar los detalles internos** de esa implementación al exterior. En otras palabras, una clase protege sus atributos (estado interno) para que solo puedan ser modificados mediante sus métodos, evitando accesos indebidos o incoherentes desde fuera.

Encapsular significa que los **atributos** de un objeto normalmente se declaran como **privados** (`private`), de modo que no sean accesibles directamente desde otros objetos. En su lugar, se proveen métodos públicos (conocidos comúnmente como *getters* y *setters*) para leer o modificar esos atributos de forma controlada. Esto asegura la **integridad** del objeto: por ejemplo, podríamos evitar que alguien asigne una edad negativa a una persona, validándolo en el setter.

Reescribamos la clase `Persona` aplicando encapsulamiento:

```java
public class Persona {
    // Atributos encapsulados (privados)
    private String nombre;
    private int    edad;

    // Constructor para facilitar la creación con valores iniciales
    public Persona(String nombre, int edad) {
        this.nombre = nombre;
        this.edad   = edad;
    }

    // Getter para leer el nombre
    public String getNombre() {
        return nombre;
    }

    // Setter para modificar el nombre
    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    // Getter para leer la edad
    public int getEdad() {
        return edad;
    }

    // Setter para modificar la edad (con una validación simple)
    public void setEdad(int edad) {
        if (edad >= 0) {                // no se permiten edades negativas
            this.edad = edad;
        }
    }

    // Método de saludo (sin cambios)
    public void saludar() {
        System.out.println("Hola, soy " + nombre + " y tengo " + edad + " años.");
    }
}
```

Imagen 3: Elaboración Propia

En esta versión, los atributos `nombre` y `edad` son privados. Solo se puede acceder a ellos a través de métodos públicos. Por ejemplo, desde fuera de la clase no podríamos hacer `persona1.edad = -5` directamente, tendríamos que usar `persona1.setEdad(-5)`, y en ese método podemos aplicar la lógica para ignorar o corregir valores inválidos. Así logramos que el objeto **controle su propio estado interno** y cómo es modificado, manteniendo la consistencia. El encapsulamiento aporta **seguridad y modularidad**, pues cambiando la implementación interna de la clase (por ejemplo, podríamos cambiar cómo se calcula algo o se almacena un atributo) no afectamos a otros componentes siempre que la interfaz pública (los métodos expuestos) se mantenga igual.

En Java, los **modificadores de acceso** (`private`, `public`, `protected`) nos permiten implementar el encapsulamiento controlando qué partes del código pueden ver o usar ciertos atributos o métodos. La buena práctica es mantener los atributos privados y exponer solo lo necesario mediante métodos públicos, o a veces `protected` si se quiere permitir acceso en subclases (algo que veremos en herencia). El encapsulamiento, en resumen, **oculta el "cómo" y expone solo el "qué"**: los objetos interactúan con otros a través de métodos públicos sin saber los detalles internos de cada uno.

Abstracción

La **abstracción** consiste en enfocarse en las **características esenciales** de algo, ignorando los detalles no relevantes para el contexto. En POO, la abstracción nos permite modelar objetos del mundo real en el software, seleccionando solo aquellos atributos y comportamientos que nos interesan y dejando de lado complejidades innecesarias. Dicho de otra forma, **abstraemos** una representación simplificada de una entidad para nuestro programa.

Por ejemplo, si queremos modelar un **Auto** en un simulador, podríamos abstraerlo con atributos como `velocidad`, `combustible` y métodos como `acelerar()` o `frenar()`. No necesitamos modelar cada tuerca del motor, porque para nuestro propósito esos detalles no son esenciales. Abstraemos la idea de "auto" con lo que importa para el software.

En Java, la abstracción se logra naturalmente al definir clases: estamos escogiendo qué atributos y métodos representan al objeto. Además, Java ofrece mecanismos específicos para la abstracción: las **clases abstractas** y las **interfaces** (que veremos más adelante). Una *clase abstracta* es una clase que no se puede instanciar directamente, pensada solo para ser heredada; suele contener métodos abstractos (sin implementación) que obligan a las subclases a definirlos,

garantizando cierto comportamiento. Las *interfaces* definen contratos de métodos que las clases que las implementen deben cumplir. Ambos mecanismos sirven para representar abstracciones en el diseño de software.

Veamos un ejemplo sencillo de **clase abstracta** para ilustrar la abstracción y cómo fuerza a las clases derivadas a definir ciertos comportamientos esenciales:

```java
// Clase abstracta Figura (representa la idea general de una figura geométrica)
public abstract class Figura {
    // Método abstracto: no tiene cuerpo, las subclases deben implementarlo
    public abstract double area();

    // Método concreto: puede tener implementación común
    public void mostrarArea() {
        System.out.println("Área = " + area());
    }
}

// Subclase concreta que extiende Figura
public class Circulo extends Figura {
    private double radio;
    public Circulo(double radio) { this.radio = radio; }
    // Implementación del método abstracto area()
    @Override
    public double area() {
        return Math.PI * radio * radio;
    }
}

// Otra subclase concreta de Figura
public class Rectangulo extends Figura {
    private double ancho;
    private double alto;
    public Rectangulo(double ancho, double alto) {
        this.ancho = ancho;
        this.alto  = alto;
    }
    @Override
    public double area() {
        return ancho * alto;
    }
}
```

Imagen 4: Elaboración Propia

En este ejemplo, `Figura` es una clase abstracta que define que *toda figura geométrica* debe ser capaz de calcular su área (`area()`), pero no especifica cómo (eso depende de cada figura concreta). `Circulo` y `Rectangulo` *heredan* de `Figura` y proveen sus propias implementaciones del método `area()`. Hemos abstraído el concepto general de "figura" con un método esencial (calcular área) sin preocuparnos en la clase base de los detalles de cada forma. La abstracción nos ayuda a manejar la complejidad, permitiendo razonar en términos generales (todas las figuras tienen área) y delegar detalles específicos a las partes correspondientes (cada subclase sabe calcular su área).

Herencia

La **herencia** es un mecanismo por el cual una clase (llamada *subclase* o clase hija) **extiende** a otra clase (llamada *superclase* o clase padre), adquiriendo automáticamente sus atributos y métodos. La herencia permite **reutilizar código** y establecer una relación jerárquica entre clases. Por ejemplo, podríamos tener una clase general `Animal` y que `Perro` y `Gato` hereden de ella, compartiendo lo común de `Animal` pero cada uno con sus particularidades. En Java se utiliza la sintaxis `extends` para indicar que una clase hereda de otra.

Al heredar, la subclase obtiene todos los atributos y métodos de la superclase (con ciertas restricciones según modificadores de acceso: los miembros declarados `private` en la superclase también son heredados pero no accesibles directamente). La subclase puede **añadir nuevos** atributos o métodos, y también puede **sobreescribir** (override) métodos de la superclase para cambiar o extender su comportamiento.

Veamos un ejemplo básico de herencia en Java:

```java
// Superclase Animal
public class Animal {
    protected String nombre;  // 'protected' permite acceso en subclases
    public Animal(String nombre) {
        this.nombre = nombre;
    }
    public void comer() {
        System.out.println(nombre + " está comiendo.");
    }
    public void hacerSonido() {
        System.out.println(nombre + " hace un sonido.");
    }
}
```

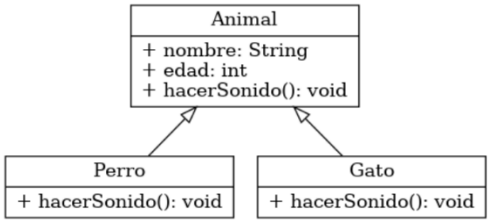
```java
// Subclase Perro que hereda de Animal
public class Perro extends Animal {
    public Perro(String nombre) {
        super(nombre); // llamar al constructor de Animal
    }
    // Sobrescribir el método hacerSonido
    @Override
    public void hacerSonido() {
        System.out.println(nombre + " dice: ¡Guau!");
    }
    // Método propio de Perro
    public void perseguirCola() {
        System.out.println(nombre + " está persiguiendo su cola.");
    }
}

// Subclase Gato que hereda de Animal
public class Gato extends Animal {
    public Gato(String nombre) {
        super(nombre);
    }
    @Override
    public void hacerSonido() {
        System.out.println(nombre + " dice: ¡Miau!");
    }
    // Método propio de Gato
    public void ronronear() {
        System.out.println(nombre + " está ronroneando.");
    }
}
```

Imagen 5: Elaboración Propia

En este ejemplo, `Animal` define atributos y métodos generales para cualquier animal. Las clases `Perro` y `Gato` usan `extends Animal` para heredar esa funcionalidad. Cada una implementa su propio sonido sobrescribiendo `hacerSonido()`, y agregan métodos específicos (`perseguirCola()` para Perro, `ronronear()` para Gato). Observa cómo usamos `super(nombre)` en los constructores de las subclases para aprovechar el constructor de Animal. También marcamos los atributos de `Animal` como `protected` para que las subclases puedan usarlos (otra opción hubiera sido `private` con getters, pero aquí `protected` simplifica el ejemplo didáctico).

Ventajas de la herencia: Podemos tratar a un `Perro` o un `Gato` como un `Animal` general cuando nos conviene (por ejemplo, para agruparlos en una lista de animales), evitando duplicar código común en cada clase. Además, la herencia fomenta modelos jerárquicos claros: un **es-un** (is-a) `Perro` es-un `Animal`, un `Gato` es-un `Animal`. La relación de herencia es representada típicamente como una **jerarquía** o árbol de clases, donde la clase base ocupa un nivel superior y las subclases se derivan debajo de ella.

*Figura: Ejemplo de diagrama UML de **herencia** en Java. La clase base `Animal` proporciona atributos (`nombre`, `edad`) y un método `hacerSonido()`, mientras que las subclases `Perro` y `Gato` heredan esas características y pueden sobreescribir el comportamiento de `hacerSonido()`.*



En el diagrama UML anterior, la flecha triangular abierta señala que `Perro` y `Gato` heredan de `Animal`. Todos los **métodos públicos** de `Animal` están disponibles en `Perro` y `Gato` (por ejemplo, podríamos llamar `perro.comer()` aunque no esté definido en `Perro` explícitamente, porque lo hereda de `Animal`). Asimismo, los atributos heredados (`nombre`, `edad`) existen en cada objeto `Perro` o `Gato`. La herencia soporta también un tipo especial de polimorfismo que veremos a continuación, y ayuda a estructurar el código evitando repetición.

📌 ***Nota***: Java **no soporta herencia múltiple de clases** (una subclase no puede extender a más de una superclase). Sin embargo, una clase sí puede implementar múltiples interfaces, lo cual proporciona cierta flexibilidad similar (lo comentaremos en la sección de *interfaces*). Todas las clases en Java extienden en última instancia de la clase base `Object` (directa o indirectamente), que proporciona métodos generales como `toString()`, `equals()`, etc.

Polimorfismo

El término **polimorfismo** significa "*muchas formas*". En POO hace referencia a la capacidad de que una referencia a una clase pueda **apuntar a objetos de distintas clases** en la jerarquía de herencia, y que invocar un método a través de esa referencia ejecute diferentes comportamientos según el *tipo real* del objeto referenciado. Dicho de forma más simple: un mismo mensaje (llamada a método) puede provocar distintas acciones dependiendo del **objeto** que lo reciba. Esto es fundamental para escribir código genérico y extensible.

Existen dos tipos de polimorfismo en Java:

- **Polimorfismo en tiempo de compilación** (también llamado *sobrecarga*, donde varios métodos comparten nombre pero difieren en parámetros) – no lo cubriremos en detalle aquí, pues es más una comodidad del lenguaje.
- **Polimorfismo en tiempo de ejecución** (basado en herencia o interfaces, también llamado *sobrescritura dinámica*), que es el más característico de la POO.

Nos enfocaremos en el polimorfismo de **sobrescritura** (runtime). Retomemos el ejemplo de `Animal`, `Perro` y `Gato`. Gracias a la herencia, un objeto `Perro` es también un tipo de `Animal`. Podemos por tanto hacer:

```
Animal a1 = new Perro("Toby");
Animal a2 = new Gato("Misu");

a1.hacerSonido();  // Imprime: "Toby dice: ¡Guau!"
a2.hacerSonido();  // Imprime: "Misu dice: ¡Miau!"
```

Imagen 6: Elaboración Propia

Aunque la variable `a1` es de tipo `Animal`, en tiempo de ejecución referencia a un objeto `Perro`. Java determinará automáticamente invocar la versión del método `hacerSonido()` propia de la clase `Perro`. Lo mismo con `a2` que realmente es un `Gato`. Este comportamiento se llama **ligadura tardía** o **dinámica** (el método invocado se decide en tiempo de ejecución según el objeto). Es un ejemplo de polimorfismo: ***un mismo método (`hacerSonido`) actuando de forma distinta según el tipo de objeto***.

El polimorfismo nos permite escribir código más genérico. Por ejemplo, podríamos tener un arreglo de `Animal` y simplemente mandar a cada uno a hacer sonido, sin preocuparnos de qué tipo específico es cada elemento:

```java
Animal[] zoologico = {
    new Perro("Rex"),
    new Gato("Minina"),
    new Perro("Firulais")
};

for (Animal animal : zoologico) {
    animal.hacerSonido();
    // Cada objeto invoca su propia versión de hacerSonido():
    // Rex dice: ¡Guau!
    // Minina dice: ¡Miau!
    // Firulais dice: ¡Guau!
}
```

Imagen 7: Elaboración Propia

Gracias al polimorfismo, podemos **tratar a objetos distintos de manera uniforme** a través de su tipo común (`Animal` en este caso), y confiar en que cada objeto se comportará adecuadamente según su clase real. Esto hace al código fácilmente extensible: si mañana agregamos una clase Vaca que extiende `Animal` y sobreescribe `hacerSonido()` (por ejemplo, para mugir), *no tenemos que cambiar el código del arreglo ni el bucle anterior*; automáticamente, una `Vaca` en ese arreglo responderá con su sonido. El código está escrito pensando en términos del *supertipo* `Animal`, y sigue funcionando para cualquier subtipo nuevo — esto es la esencia del polimorfismo.

Polimorfismo e interfaces: El polimorfismo no solo aplica a clases en relación de herencia. También funciona mediante **interfaces**. Si varias clases implementan la misma interfaz (digamos, `Volable` con método `volar()`), podemos referirnos a objetos de esas clases via la interfaz y llamar a `volar()` independientemente de su clase concreta. Este uso polimórfico de interfaces es muy poderoso en Java, como veremos ahora.

Interfaces

Una **interfaz** en Java es un conjunto de métodos (y constantes) que actúa como un **contrato**: las clases que *implementan* esa interfaz se comprometen a proporcionar una implementación para cada método definido en la interfaz. Las interfaces ayudan a lograr abstracción y polimorfismo, ya que permiten definir capacidades que distintas clases no relacionadas pueden compartir. A diferencia de la herencia de clases, donde una subclase solo puede heredar de una superclase, una clase en Java **puede implementar múltiples interfaces**. Esto permite, por ejemplo, que una clase cumpla varios contratos de comportamiento diferentes.

Podemos pensar en una interfaz como en un rol o habilidad. Por ejemplo, podríamos tener una interfaz `Volador` que define el método `volar()`. Varias clases pueden *tener la capacidad de volar* implementando esa interfaz, incluso si no comparten una relación de herencia directa. Un `Avión` y un `Pájaro` no heredan uno del otro, pero ambos pueden implementar `Volador` y por tanto ser tratados como "cosas que vuelan".

Definamos una interfaz sencilla y dos clases que la implementen:

```java
// Definición de la interfaz
interface Sonido {
    void sonar();  // método abstracto (implícitamente público y abstracto)
}

// Clase que implementa la interfaz Sonido
class Guitarra implements Sonido {
    @Override
    public void sonar() {
        System.out.println("Guitarra suena: Strum strum...");
    }
}

// Otra clase que implementa Sonido
class Claxon implements Sonido {
    @Override
    public void sonar() {
        System.out.println("Claxon suena: ¡Beep Beep!");
    }
}
```

Imagen 8: Elaboración Propia

Aquí `Sonido` es una interfaz que define un método `sonar()`. Tanto `Guitarra` como `Claxon` la implementan, proporcionando cada una su versión de cómo suenan. Gracias a esto, podemos utilizar el polimorfismo con la interfaz:

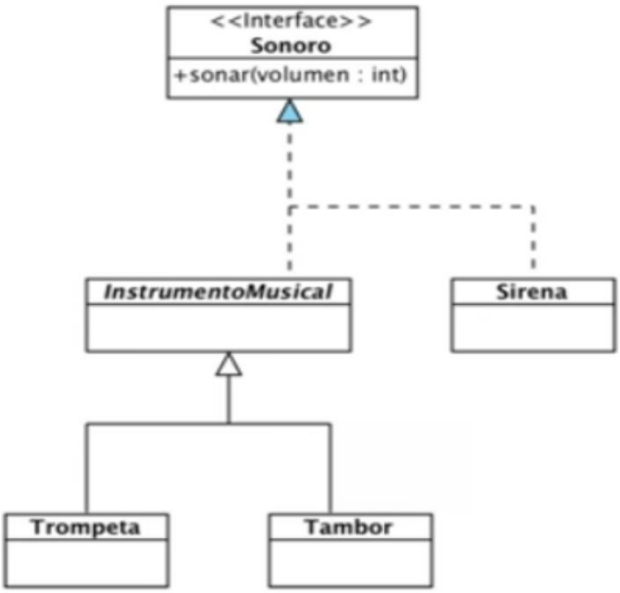
```
Sonido algoSonoro = new Guitarra();
algoSonoro.sonar();   // Salida: "Guitarra suena: Strum strum..."

algoSonoro = new Claxon();
algoSonoro.sonar();   // Salida: "Claxon suena: ¡Beep Beep!"
```

Imagen 9: Elaboración Propia

La variable `algoSonoro` es de tipo `Sonido` (la interfaz), pero puede referenciar a cualquier objeto de una clase que implemente esa interfaz. En tiempo de ejecución, se llama al método apropiado según si es una guitarra o un claxon — polimorfismo a través de interfaz.

Las **interfaces** se usan mucho en Java para definir *lo que algo puede hacer* sin importar *qué es en sí*. Por ejemplo, la API de Java tiene la interfaz `List` que define operaciones de una lista (agregar, quitar, obtener elemento...), y múltiples implementaciones (como `ArrayList`, `LinkedList`) que cumplen ese contrato. Cuando escribimos código, típicamente manejamos la variable como `List` genérica, y podemos cambiar la implementación concreta sin afectar al resto del programa.

Figura: Diagrama de clases que ilustra el uso de una interfaz `Sonoro` implementada por dos clases (InstrumentoMusical y Sirena). La interfaz define el método `sonar(volumen:int)` que ambas clases deben implementar. Además, se muestra que `Trompeta` y `Tambor` heredan de `InstrumentoMusical`. Las líneas de puntos indican la relación "realiza/implementa interface".



En el diagrama anterior, `Sonoro` es una interfaz (indicada por «<<Interface>>») que es implementada tanto por `InstrumentoMusical` como por `Sirena`. Esto significa que objetos de esas clases pueden ser tratados polimórficamente como `Sonoro`. A su vez, `InstrumentoMusical` tiene subclases (`Trompeta` y `Tambor`), pero todas ellas comparten que pueden sonar. El diseño con interfaces tiende puentes entre clases que de otro modo no estarían relacionadas en la jerarquía de herencia.

Diferencia entre clase abstracta e interfaz: Una clase abstracta puede proporcionar cierta implementación y mantener estado (atributos), mientras que una interfaz en Java tradicionalmente no tenía implementación (en Java 8+ se permitieron métodos *default* con implementación en interfaces, pero siguen sin poder tener estado, aparte de constantes). Además, una clase puede implementar muchas interfaces pero solo extender de una clase. Elegir entre usar una clase abstracta o una interfaz depende del problema: si realmente hay una relación de tipo "es-un" y algo de comportamiento por defecto, una clase base abstracta puede convenir; si solo se desea definir una capacidad sin forzar una jerarquía, la interfaz es la opción. A veces se combinan: se define una interfaz y una clase abstracta que la implementa parcialmente, para que otras clases hereden de la abstracta.

Principios de Diseño SOLID

Además de los conceptos fundamentales de la POO, existen una serie de **principios de diseño** que ayudan a escribir código orientado a objetos que sea más mantenible, extensible y robusto. Cinco de los más reconocidos son los **principios SOLID**, acrónimo que corresponde a:

- **S** – *Single Responsibility Principle* (**Principio de Responsabilidad Única**)
- **O** – *Open/Closed Principle* (**Principio Abierto/Cerrado**)
- **L** – *Liskov Substitution Principle* (**Principio de Sustitución de Liskov**)
- **I** – *Interface Segregation Principle* (**Principio de Segregación de Interfaces**)
- **D** – *Dependency Inversion Principle* (**Principio de Inversión de Dependencias**)

Formulados inicialmente por Robert C. Martin (conocido como "Uncle Bob") a principios de la década de 2000, estos principios buscan guiar al desarrollador para evitar diseños deficientes y favorecer buenas prácticas. A continuación, explicaremos cada principio de forma sencilla con algún ejemplo representativo en Java.

Principio de Responsabilidad Única (Single Responsibility Principle)

Este principio establece que **una clase (o módulo) debe tener una única responsabilidad**, es decir, una única razón para cambiar. En la práctica, significa que cada clase debe encargarse de hacer *solo una cosa* y hacerlo bien. Si una clase tiene más de una responsabilidad, conviene separarla en clases diferentes. De este modo, nuestro diseño será más claro y, cuando haya que modificar algo, será más fácil localizar dónde hacerlo sin afectar otras partes.

Ejemplo: Imaginemos una clase `Reporte` que a la vez calcula datos **y** los muestra por pantalla. Tendríamos dos motivos diferentes para cambiar esta clase: 1) si cambia la forma de cálculo, y 2) si queremos cambiar la forma de presentar el resultado. Esto viola el principio de responsabilidad única. La solución sería dividir esas tareas en dos clases: una clase que calcule o genere el reporte (lógica de negocio) y otra clase que se encargue de la presentación (por ejemplo, formatear e imprimir el reporte). Así, cada clase tiene su razón de cambio independiente: la de cálculo solo cambiaría si varían las reglas de cálculo, y la de impresión solo si cambia el formato o medio de salida.

En código, podría ser algo así:

```java
// Ejemplo problemático: Una clase con dos responsabilidades
class Reporte {
    public String generarResumen() {
        // ... (lógica para generar datos del resumen)
        return "Resumen de datos...";
    }
    public void imprimirResumenConsola() {
        String resumen = generarResumen();
        System.out.println("**** REPORTE ****");
        System.out.println(resumen);
    }
}
```

Imagen 10: Elaboración Propia

La clase `Reporte` anterior hace dos cosas: genera el contenido y lo imprime en consola. Aplicando responsabilidad única, la separamos:

```java
class GeneradorReporte {
    public String generarResumen() {
        // ... (lógica para generar datos del resumen)
        return "Resumen de datos...";
    }
}

class ImpresoraReporte {
    public void imprimirEnConsola(String contenido) {
        System.out.println("**** REPORTE ****");
        System.out.println(contenido);
    }
}
```

Imagen 11: Elaboración Propia

Ahora `GeneradorReporte` solo crea los datos, e `ImpresoraReporte` solo los imprime. Si en el futuro queremos imprimir el reporte en un archivo o PDF, podemos añadir otro método o clase impresora sin tocar la generación del contenido. Cada clase tiene un propósito claro y único. Esto aumenta la cohesión del código (partes bien enfocadas) y reduce el riesgo de efectos colaterales al modificar.

En términos generales, cuando notemos que estamos usando muchas palabras "y" al describir la responsabilidad de una clase (ejemplo: "esta clase calcula X **y** también muestra Y **y** además valida Z"), es una señal de que quizás deba refactorizarse en varias clases.

Principio Abierto/Cerrado (Open/Closed Principle)

El principio Abierto/Cerrado propone que las entidades de software (clases, módulos, funciones) deben estar **abiertas para extensión, pero cerradas para modificación**. Esto significa que deberíamos poder agregar nuevas funcionalidades o casos de uso **sin tener que modificar el código existente** de esas entidades, evitando así introducir errores en lo ya probado.

En la práctica, este principio nos anima a usar la **herencia, interfaces y polimorfismo** para agregar comportamientos nuevos, en lugar de alterar código ya escrito. Un enfoque típico es evitar largas estructuras condicionales que verifican tipos para decidir qué hacer (porque cada vez que agreguemos un tipo nuevo, tendríamos que modificar esa estructura). En cambio, delegamos la responsabilidad a nuevas clases que extiendan o implementen comportamientos.

Ejemplo clásico – cálculo de área de figuras: Supongamos una función que calcula el área de una forma geométrica, donde la forma puede ser un círculo o un rectángulo. Una **mala** implementación que viola Abierto/Cerrado sería:

```
public double calcularArea(Object forma) {
    if (forma instanceof Circulo) {
        Circulo c = (Circulo) forma;
        return Math.PI * c.radio * c.radio;
    } else if (forma instanceof Rectangulo) {
        Rectangulo r = (Rectangulo) forma;
        return r.ancho * r.alto;
    }
    return 0;
}
```

Imagen 12: Elaboración Propia

Si luego queremos agregar un **Triángulo**, tendríamos que *modificar* esta función para incluir otro else if para triángulos. Cada nueva figura implica tocar este código (riesgo de error y violación de cerrado a modificación). En su lugar, apliquemos polimorfismo:

```java
abstract class Figura {
    public abstract double area();
}

class Circulo extends Figura {
    double radio;
    public Circulo(double r) { radio = r; }
    @Override
    public double area() {
        return Math.PI * radio * radio;
    }
}

class Rectangulo extends Figura {
    double ancho, alto;
    public Rectangulo(double a, double b) { ancho = a; alto = b; }
    @Override
    public double area() {
        return ancho * alto;
    }
}

// (Podríamos fácilmente agregar más figuras: Triangulo, Cuadrado, etc., implementando area())
```

Imagen 13: Elaboración Propia

Con este diseño, **extender** el sistema con una nueva figura (p.ej. Triangulo) solo requiere crear una nueva subclase que implemente el método area(). No necesitamos modificar la clase Figura ni cambiar código en las clases existentes. El método de cálculo podría ser tan simple como:

```java
public double calcularArea(Figura f) {
    return f.area();
}
```

Imagen 14: Elaboración Propia

hora está *abierto* a calcular áreas de cualquier nueva Figura que creemos, y *cerrado* a modificaciones: no hace falta reescribirlo al añadir nuevas formas.

En general, para seguir Open/Closed, piensa en dónde tu código puede necesitar soportar variantes en el futuro, y diseña esas partes de forma polimórfica. Usa clases base, interfaces, y evita escribir código que discrimine tipos manualmente. Así, incorporas nuevas funcionalidades agregando código nuevo (extensión) en vez de tocar el ya escrito.

Principio de Sustitución de Liskov (Liskov Substitution Principle)

Formulado por Barbara Liskov, este principio indica que **una clase derivada debe poder usarse en lugar de su clase base sin romper la lógica del programa**. En otras palabras, **objetos de una superclase deben poder ser reemplazados por objetos de sus subclases** en cualquier contexto, *y el programa debería seguir funcionando correctamente*. Si una subclase no cumple con las expectativas de comportamiento establecidas por la superclase, entonces no es realmente un subtipo válido, y estamos violando este principio.

El LSP se trata de preservar las **contratos**: la interfaz pública y el comportamiento esperado de la superclase deben mantenerse en las subclases. Si la superclase promete algo mediante sus métodos, la subclase no debe alterar esas promesas. Por ejemplo, si tenemos una clase `Vehiculo` con un método `mover(int distancia)` que supone siempre mover hacia adelante la distancia positiva dada, una subclase CocheAutomatico no debería reinterpretar `mover(10)` como moverse 10 hacia adelante a veces o hacia atrás otras, porque rompería la expectativa.

Ejemplo clásico – Rectángulo/Cuadrado: Supongamos una clase `Rectangulo` con métodos `setAncho(int)` y `setAlto(int)` y un método `getArea()`. Uno podría pensar que un **Cuadrado** es un caso particular de rectángulo y hacer que `Cuadrado` herede de `Rectangulo`. Sin embargo, en términos de comportamiento, esto puede violar LSP. Veamos por qué:

```
class Rectangulo {
    protected int ancho, alto;
    public void setAncho(int ancho) { this.ancho = ancho; }
    public void setAlto(int alto)   { this.alto = alto; }
    public int getArea() { return ancho * alto; }
}

class Cuadrado extends Rectangulo {
    @Override
    public void setAncho(int ancho) {
        this.ancho = ancho;
        this.alto  = ancho;  // asegura que ancho == alto
    }
    @Override
    public void setAlto(int alto) {
        this.alto  = alto;
        this.ancho = alto;   // asegura que ancho == alto
    }
}
```

Imagen 15: Elaboración Propia

La implementación de Cuadrado fuerza que *ancho* y *alto* siempre sean iguales (porque eso define un cuadrado). En apariencia, esto funciona, pero observemos una situación:

```
Rectangulo r = new Cuadrado();
r.setAncho(5);
r.setAlto(4);
// ¿Qué esperamos aquí?
System.out.println("Área: " + r.getArea());
```

Imagen 16: Elaboración Propia

Si `r` es un `Rectangulo` normal, tras `setAncho(5)` y `setAlto(4)` esperaríamos un rectángulo de 5x4 con área 20. Pero como en realidad es un `Cuadrado` (instancia de subclase), la implementación de `setAlto` habrá ajustado también el ancho a 4. El área calculada será 16, no 20. El código que usa `Rectangulo` quedó sorprendido: la *sustitución* de un Rectángulo por un Cuadrado **rompió las expectativas**. Esto significa que `Cuadrado` no cumple completamente con el contrato de `Rectangulo` (sus métodos no se comportan de forma 100% compatible). En consecuencia, a pesar de la relación matemática entre cuadrado y rectángulo, en diseño de software no es aconsejable modelarlos con herencia directa, porque viola LSP.

El Principio de Liskov nos anima a diseñar jerarquías de herencia donde las subclases **no alteren las reglas básicas** de la superclase. Algunas guías para lograr esto:

- No debilitar las condiciones preestablecidas por la superclase en los métodos sobrescritos, ni las postcondiciones. Por ejemplo, si un método de la superclase espera un parámetro no nulo, la subclase no debería aceptar nulos (precondición más débil).
- No cambiar la interpretación de los datos. (Como en el caso cuadrado/rectángulo, la interpretación de ancho/alto cambió).
- Asegurarse de que cualquier propiedad lógica que se cumpla para la superclase siga cumpliéndose para la subclase. Si `Rectangulo` asumía que después de `setAncho(a)` el ancho permanece siendo a independientemente de la altura, en `Cuadrado` eso no se da si después se cambia la altura.

Cuando notamos que una subclase no puede integrarse limpiamente en el diseño donde se espera a su superclase, debemos reconsiderar si la relación de

herencia es correcta. Tal vez la subclase debería ser independiente o la jerarquía reorganizada. Mantener LSP suele implicar a veces restringir la herencia a casos donde realmente hay una relación *es-un* sin excepciones extrañas, o diseñar las clases base con suficiente abstracción para que las subclases encajen.

En resumen, **siempre que podamos usar un objeto de la superclase, deberíamos poder usar cualquier subclase suya sin notar diferencias en la corrección del programa**. Cumplir con esto asegura jerarquías consistentes y evitan sorpresas en el comportamiento.

Principio de Segregación de Interfaces (Interface Segregation Principle)

El principio de Segregación de Interfaces sugiere que es mejor tener **muchas interfaces específicas y pequeñas** en lugar de una única interfaz grande y "gorda" que obligue a las clases a implementar métodos que no usan. Dicho de otra forma, **no debemos forzar a una clase a depender de métodos que no necesita**. Este principio tiene sentido porque las interfaces son contratos: si son demasiado amplios, cualquier clase que las implemente estará aceptando una responsabilidad mayor de la que le corresponde.

Ejemplo – Interfaces demasiado grandes: Imagina una interfaz `Ave` que define métodos `volar()`, `nadar()` y `caminar()`. Podría parecer razonable si pensamos en general sobre aves. Sin embargo, un pingüino es un ave que *no vuela*. Si `Pingüino` implementa `Ave`, se ve forzado a proporcionar una implementación de `volar()` que realmente no tiene sentido (quizá dejarlo vacío o lanzar una excepción). Esto es un diseño deficiente. El ISP nos indica que dividamos esa interfaz en interfaces más pequeñas y específicas de cada habilidad:

```
interface Volador {
    void volar();
}
interface Nadador {
    void nadar();
}
interface Caminante {
    void caminar();
}
```

Imagen 17: Elaboración Propia

Ahora podemos tener clases que implementen solo lo que necesitan:

```
class Aguila implements Volador, Caminante {
    public void volar() { /* ... */ }
    public void caminar() { /* ... */ }
}
class Pinguino implements Nadador, Caminante {
    public void nadar() { /* ... */ }
    public void caminar() { /* ... */ }
}
```

Imagen 18: Elaboración Propia

`Aguila` implementa volar y caminar, `Pinguino` implementa nadar y caminar. Ninguna está obligada a implementar un método inútil para ellas. Si en el futuro aparece un ave que también vuela y nada (digamos un ave ficticia acuática), podría implementar las interfaces que correspondan.

La idea es diseñar **interfaces enfocadas en un rol específico**. En Java es común ver interfaces pequeñas como `Iterable` (solo define un iterador), `Closeable` (solo define un método close), etc. Estas interfaces segregadas permiten que las clases las combinen según necesiten, sin peso muerto.

Otro ejemplo – caso práctico: Supongamos que tenemos una interfaz amplia para manejo de datos:

```
interface BaseDeDatos {
    void guardar(Object o);
    void eliminar(Object o);
    void abrirConexion();
    void cerrarConexion();
    void realizarBackup();
    // etc...
}
```

Imagen 19: Elaboración Propia

Y tenemos clases que implementan esta interfaz. Quizá una clase `BaseDatosMemoria` que guarda en estructura de datos en memoria no necesita `abrirConexion()` ni `cerrarConexion()` (porque no hay conexión real) ni hacer backup. Sin embargo, tiene que implementarlos (tal vez vacíos) por el contrato. Esto genera código confuso e interfaces con métodos que no siempre aplican. Mejor podríamos separar responsabilidades: una interfaz para operaciones CRUD básicas (`RepositorioDatos` con métodos guardar, eliminar, buscar), otra interfaz para las operaciones de conexión (`ConexionBD` con abrir/cerrar), otra para backup

(`Backupable` con `realizarBackup`). De esa forma, una base de datos en memoria implementaría solo RepositorioDatos, mientras que una base de datos SQL implementaría `RepositorioDatos` y `ConexionBD` y `Backupable` si corresponde. Cada quien implementa solo lo necesario.

En resumen, **Interface Segregation** nos guía a dividir interfaces grandes en varias más pequeñas cuando sea posible. Así, las clases mantienen sus dependencias al mínimo requerido. Esto produce un sistema más flexible y comprensible, ya que cada interfaz representa un papel específico.

Principio de Inversión de Dependencias (Dependency Inversion Principle)

El principio de Inversión de Dependencias sostiene dos reglas relacionadas

1. Los **módulos de alto nivel** (aquellos que definen la lógica de negocio de alto nivel) **no deberían depender de módulos de bajo nivel** (los detalles de implementación); **ambos deberían depender de abstracciones**.
2. Las **abstracciones no deberían depender de los detalles**; más bien, **los detalles (concretos) deberían depender de abstracciones**.

En términos sencillos, DIP nos pide que **desacoplemos** las partes generales de las específicas mediante abstracciones. En lugar de que una clase de alto nivel *controle* a las clases concretas de bajo nivel, volteamos la relación: definimos interfaces (abstracciones) que las clases de bajo nivel implementan, y la clase de alto nivel usa dichas interfaces. Así, la clase de alto nivel no sabe nada de las clases concretas, solo conoce la interfaz (o clase abstracta) con la que trabaja.

Este principio a menudo se implementa mediante la técnica de **inyección de dependencias**: en vez de que una clase cree por sí misma sus dependencias, se las suministramos externamente (por constructor, métodos set o un contenedor IoC). De esa manera, podemos pasarle diferentes implementaciones sin cambiar la clase de alto nivel.

Ejemplo – sin DIP (acoplamiento fuerte): Supongamos que tenemos una clase `Coche` que usa un motor:

```
class MotorGasolina {
    public void encender() { System.out.println("Motor de gasolina encendido"); }
}

class Coche {
    private MotorGasolina motor = new MotorGasolina();
    public void arrancar() {
        motor.encender();
    }
}
```

Imagen 20: Elaboración Propia

Aquí `Coche` depende directamente de una clase concreta `MotorGasolina` (módulo de bajo nivel). Si quisiéramos que el coche use un `MotorElectrico`, tendríamos que **modificar** la clase `Coche` para cambiar la instancia de motor. Esto está fuertemente acoplado y es rígido.

Aplicando **Dependency Inversion**, introducimos una abstracción `Motor` (una interfaz, por ejemplo) de la cual dependa `Coche`:

```
// Abstracción (interfaz) para el motor
interface Motor {
    void encender();
}

// Implementación concreta 1
class MotorGasolina implements Motor {
    public void encender() {
        System.out.println("Motor de gasolina encendido");
    }
}

// Implementación concreta 2
class MotorElectrico implements Motor {
    public void encender() {
        System.out.println("Motor eléctrico encendido");
    }
}

// Clase de alto nivel que depende de la abstracción Motor, no de concreciones
class Coche {
    private Motor motor;
    // El motor se inyecta via el constructor (Inyección de dependencias)
    public Coche(Motor motor) {
        this.motor = motor;
    }
    public void arrancar() {
        motor.encender();
    }
}
```

Imagen 21: Elaboración Propia

Ahora Coche no crea el motor, ni sabe si es de gasolina o eléctrico; solo conoce la interfaz Motor. Podemos instanciar Coche con el tipo de motor que necesitemos:

```
MotorGasolina mGas = new MotorGasolina();
Coche auto1 = new Coche(mGas);
auto1.arrancar();  // imprime "Motor de gasolina encendido"

MotorElectrico mElec = new MotorElectrico();
Coche auto2 = new Coche(mElec);
auto2.arrancar();  // imprime "Motor eléctrico encendido"
```

Imagen 22: Elaboración Propia

Hemos **invertido la dependencia**: antes Coche (alto nivel) dependía de MotorGasolina (bajo nivel); ahora Coche depende de la *interfaz* Motor, y son las implementaciones concretas MotorGasolina/MotorElectrico las que dependen de la interfaz también (la implementan). Si mañana definimos un MotorHibrido, no necesitamos tocar la clase Coche en absoluto, solo crear la nueva clase que implementa Motor y pasarla al coche. Esto reduce el acoplamiento, mejora la **modularidad** y facilita la **extensión** y **pruebas unitarias** (ya que podríamos incluso tener una implementación de Motor simulada para pruebas).

En entornos empresariales de Java, este principio se aplica masivamente usando *frameworks* de inyección de dependencias (como Spring) que automáticamente pasan las implementaciones apropiadas a las clases, manteniendo todo desacoplado. Pero incluso sin frameworks, es un buen patrón de diseño diseñar nuestras clases pensando en abstracciones e invertir las dependencias de modo que las piezas de alto nivel no estén atadas a los detalles.

Ejemplo SOLID

Imaginemos un **restaurante de alta cocina** donde cada proceso está cuidadosamente diseñado para ofrecer una experiencia impecable. Este escenario nos permite visualizar cómo se aplican todos los principios SOLID de manera integrada:

1. **Responsabilidad Única (SRP):** Cada empleado del restaurante tiene un rol específico y bien definido.
    - El **mesero** se encarga de tomar pedidos y atender a los clientes.
    - El **chef** prepara los platos siguiendo las recetas.
    - El **gerente** supervisa el funcionamiento general sin meterse en las tareas operativas.

    Así, cada función se realiza por quien está especializado en ella, evitando que un empleado tenga que cumplir múltiples roles y complicar el sistema.

2. **Abierto/Cerrado (OCP):** El restaurante está diseñado para poder **extender** sus servicios sin modificar lo que ya funciona.
    - Por ejemplo, si se decide incluir un **bartender** para preparar cócteles, se añade a la estructura sin alterar los roles ya existentes (mesero, chef, gerente).
    - El menú se amplía con nuevas recetas, pero las reglas de atención y preparación siguen intactas.

    De esta forma, el sistema es "abierto" a nuevas extensiones (nuevos roles o platos) pero "cerrado" a modificaciones que puedan romper la operación actual.

3. **Sustitución de Liskov (LSP):** Si se cambia a un **mesero experimentado** por uno recién contratado, la experiencia del cliente y el funcionamiento del sistema no se ven afectados, siempre que el nuevo mesero cumpla con el contrato esperado (tomar pedidos, servir correctamente).
    - Es decir, cualquier mesero, sin importar su estilo o experiencia, puede sustituir a otro sin alterar el proceso general del restaurante.

    Esto asegura que las funciones esenciales no se vean comprometidas al reemplazar a un empleado por otro que respete las mismas reglas de servicio.

4. **Segregación de Interfaces (ISP):** Las tareas del restaurante se organizan en **interfaces o áreas de responsabilidad bien definidas**.
    - Por ejemplo, se tienen protocolos específicos para atención al cliente, preparación de alimentos y manejo de pagos.
    - Un mesero no necesita conocer los detalles de cómo se cocina un plato, solo cómo presentar el menú y recibir pedidos.
    - De la misma manera, el chef se concentra en recetas y técnicas culinarias sin preocuparse por la atención al cliente.

    Así, cada "interfaz" de servicio es pequeña y enfocada, y los empleados solo implementan las funciones que les son relevantes.

5. **Inversión de Dependencias (DIP):** La **gerencia del restaurante** no interactúa directamente con cada detalle de cómo toma pedidos el mesero o prepara los platos el chef.
    - En cambio, el gerente depende de **abstracciones** (por ejemplo, protocolos o estándares de servicio) que definen el comportamiento esperado sin conocer la implementación exacta.
    - Esto permite que, si se decide cambiar la forma de operar (como introducir un sistema digital para pedidos), el gerente sigue supervisando el servicio sin tener que modificar su forma de gestionar el restaurante.

    Así, las dependencias van de lo concreto a lo abstracto, facilitando cambios en los detalles operativos sin afectar la administración general.

En este restaurante, cada empleado cumple con una única responsabilidad, lo que facilita el cambio y la mejora de cada área sin afectar a las demás (SRP). El sistema permite añadir nuevos roles o ampliar el menú sin necesidad de modificar la estructura existente (OCP). Cualquier empleado que cumpla con las funciones requeridas puede sustituir a otro sin alterar el flujo de trabajo (LSP). Las funciones están divididas en interfaces o protocolos especializados, de modo que cada empleado implementa solo lo necesario para su rol (ISP). Finalmente, la dirección del restaurante se basa en estándares y protocolos (abstracciones), lo que les permite operar de forma independiente de los detalles de la ejecución de cada tarea (DIP).

Este ejemplo ilustra cómo, en la vida cotidiana, un sistema bien organizado —como el de un restaurante— integra todos los principios SOLID para lograr eficiencia, flexibilidad y robustez. Cada cambio o extensión se hace de manera controlada, manteniendo la calidad y el buen funcionamiento del sistema en conjunto.

Conclusión

La Programación Orientada a Objetos en Java es un paradigma poderoso que, bien aplicado, permite manejar la complejidad del software de forma más natural, segura y mantenible. Hemos visto los conceptos básicos: **clases** como plantillas, **objetos** como instancias concretas, el **encapsulamiento** para proteger datos, la **abstracción** para modelar lo esencial, la **herencia** para reutilizar código estableciendo jerarquías, el **polimorfismo** para tratar objetos de forma genérica, y las **interfaces** para definir contratos flexibles. Cada concepto viene acompañado de prácticas recomendadas, como usar modificadores de acceso apropiadamente, preferir la composición e interfaces múltiples en vez de herencias inapropiadas, etc.

Asimismo, exploramos los principios **SOLID**, que sirven como guía para diseñar clases y relaciones orientadas a objetos de calidad: una clase, una responsabilidad; código abierto a extenderse, pero estable una vez escrito; subtipos que realmente se comportan como sus tipos base; interfaces específicas para no imponer cargas extras; y dependencias invertidas hacia abstracciones para lograr desacoplamiento. Estos principios, acompañados de ejemplos, muestran cómo podemos escribir código Java más limpio, modular y fácil de mantener.

Atribución-No Comercial-Sin Derivadas
Se permite descargar esta obra y compartirla, siempre y cuando no sea modificado y/o alterado su contenido, ni se comercialice. Universidad Tecnológica Nacional Facultad Regional Córdoba (S/D). Material para la Tecnicatura Universitaria en Programación, modalidad virtual, Córdoba, Argentina.